HDOC Day-64

Name: Tiksha **SAP ID:** 590034114

Problem

Write a C program to read the size and elements of a variable-length integer array **nums[]**. Exactly one value occurs twice and every other value occurs once. Pass the array to **duplicate_element()** using a pointer and display the duplicate in a single traversal.

C Program

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int value;
    struct Node *next;
} Node;

void duplicate_element(int *nums, int n) {
    int size = 2 * n + 1;
    Node **table = calloc(size, sizeof(Node *));
    if (table == NULL) return;

    for (int i = 0; i < n; i++) {
        int value = *(nums + i);
        int index = (unsigned int)value % size;
        Node *p = table[index];
        while (p != NULL) {
            if (p->value == value) {
                printf("Repeated element: %d\n", value);
                goto cleanup;
            }
            p = p->next;
        }
        Node *node = malloc(sizeof(Node));
        if (node == NULL) goto cleanup;
        node->value = value;
        node->next = table[index];
        table[index] = node;
    }
cleanup:
    for (int i = 0; i < size; i++) {
        Node *p = table[i];
        while (p != NULL) {
            Node *next = p->next;
            free(p);
            p = next;
        }
    }
    free(table);
}

int main(void) {
    int n;
```

```
    printf("Enter array size: ");
    if (scanf("%d", &n) != 1 || n < 2) return 1;
    int nums[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);
    duplicate_element(nums, n);
    return 0;
}
```

Sample Output

```
Enter array size: 5
Enter 5 elements: 4 2 7 2 9
Repeated element: 2
```

Explanation

The array is passed by its address to duplicate_element(). A hash table stores values already encountered. During one traversal, the first value found in the table is the repeated element. Expected time complexity is O(n), with O(n) additional memory.

Note: XOR alone cannot identify the duplicate in an arbitrary array of otherwise distinct integers. It works when the array is known to contain all numbers from 1 to n-1, with exactly one duplicate.